

The QA Agent — writing, running, repairing and reporting the tests

A test-authoring, test-running, test-repairing and browser-driving subsystem that rides alongside app generation, writes Vitest unit tests per file while the app is still being built, runs and repairs them under a snapshot-backed revert loop, drives the finished app through a real Chromium with Playwright, and leaves four machine-readable artifacts on disk that feed a nine-tab studio view and a PDF export.

Repository	https://github.com/Ravindu200232/full-stack-ai-web-developer
Branch	qa-agent
Scope	qa_agent/ and studio/components/testing/
Written	2026-08-14

Contents

Contents	2
1. Overview	3
2. What it does	3
3. Technology and tools	7
4. How it works	9
1. 1. Bind, without touching the build	9
2. 2. Files land; batches are queued	9
3. 3. One model call per batch writes the tests	9
4. 4. Three Python passes, then disk	9
5. 5. The author verifies its own batch	9
6. 6. Drain, backfill, staleness	9
7. 7. The unit stage	10
8. 8. Arbitration and the write allowlist	10
9. 9. Reverting, stalling, escalating	10
10. 10. The final accounting	10
11. 11. The browser pass	10
12. 12. Sweep, security, performance, report	10
5. The code, file by file	11
6. Why it is built this way	13
7. Interfaces	15

1. Overview

The QA agent is 15 files and 6,337 lines under ``qa_agent/``, plus ten React components under ``studio/components/testing/``, plus roughly 900 lines of orchestration inside ``server.py``. Its organising idea, stated in ``qa_agent/__init__.py``, is that the build pipeline must not be touched: ``QASession`` wraps two callbacks the architect already fires (``on_phase`` and ``on_file_written``), calls the original first and enqueues afterwards, and swallows every exception — so a broken QA agent cannot stop a build. Every model call it makes builds its own message list and hands it to ``arch._stream(convo, ...)``, so the build conversation is read from and never written to. Generated code is always read from disk, never from ``arch.files``, because the generation thread is mutating that dict while QA runs.

The second organising idea is a split of authority: ``**AgentForge`` writes the configuration and the mocks; the model writes only the test bodies. ``qa_agent/harness.py`` emits ``vite.config.mjs``, ``tests/setup.js`` and four helper modules — ``mongoMock.js``, ``authMock.js``, ``navMock.js``, ``request.js``, plus a ``nextLink.jsx`` alias — by hand, in Python, because every hazard in this space (Vitest not reading ``jsconfig.json``'s ``@/`` alias, ``vi.mock`` hoisting, ``new ObjectId('123')`` throwing, ``@/lib/mongodb`` connecting at import) produces a failure that reads exactly like an application bug, at which point the repair loop goes after correct code. ``mongoMock.js`` is not a stub but a small in-memory query engine: it implements the operators routes actually use — including ``updateMany`` and ``bulkWrite``, whose absence used to surface as a 500 from the route's own try/catch — and ``**throws by name`` on the rest, because a mock that silently returns nothing points the reader at the app instead of at itself. Which file is worth testing at all is also decided in Python: ``qa_agent/spec.py`` classifies each generated file into four tiers and hard-excludes async Server Components (Next's own bundled docs say Vitest cannot render them), the Mongo/auth boundary modules, and everything AgentForge itself generates.

Running is ``qa_agent/runner.py``: ``npx vitest run --reporter=json --outputFile=...`` writing to ``agentforge/qa/vitest.json``, never to stdout, because ``CommandRunner`` truncates output at 4000 characters and a truncated JSON document is unparseable. Every failure is mapped back through the manifest to the application file it is about, which is what makes a repair allowlist possible at all. Failures are classified in Python — ``IMPORT``, ``SYNTAX``, ``TIMEOUT``, ``ASSERTION``, ``RUNTIME`` — and given a human diagnosis from thirteen measured patterns; the rendered DOM is split out of the blob and compacted, and two failure classes whose own message says nothing useful (a frozen-clock timeout, an ``ObjectId-versus-string`` comparison) get a written explanation carried in a dedicated ``hint`` field rather than appended to the stack, because the stack is truncated at 2000 chars and every prompt builder filters it to lines beginning ``at ``.

The repair half is deliberately not here: ``agents/bugfixer.py`` arbitrates whether the test or the code is wrong under a ``VERDICT :: test|code :: ...`` protocol whose default is ``test``. What ``server.py`` owns is the loop around it — up to four rounds, four files repaired in parallel, a ``FileSnapshot`` capturing the bytes of every file a round will touch, a revert when the round did not strictly reduce failures, two stalls escalating through four repair tiers, and a hard rule that a code fix must survive a rebuild or be restored byte for byte. Nothing is ever silenced: ``_leave_unresolved`` explicitly replaced an older behaviour that rewrote red cases as ``it.skip`` and moved whole files to the quarantine directory, so that "the suite is green" became true by deletion. The stage now separately counts ``it.skip`` cases that never ran and cases that existed at the start and are gone from the suite, and reports both as not passing.

2. What it does

Feature	What it does	Where
Deterministic target selection, four tiers	<code>`select_targets`</code> decides what is worth a unit test without asking a model: tier 1 route handlers, tier 2 pure components, tier 3 plain lib modules, tier 4 interactive client components. Hard exclusions carry their reason — anything under <code>`app/`</code> that is not <code>`route.js`</code> is an async Server Component Vitest cannot render, <code>`lib/mongodb.js`</code> / <code>`lib/auth.js`</code> connect at module scope, and everything in the <code>`GENERATED`</code> frozenset is AgentForge's own scaffold that the model may not edit. Capped at 4 per phase and 30 per project, sorted best-tier-first.	<code>qa_agent/spec.py</code>
Authoring that overlaps generation	<code>`QASession.on_file_written`</code> fires the moment a file lands, buffers until there are four, and enqueues a batch; <code>`on_phase`</code> catches whole phases as a fallback. On a cloud QA model up to four author threads run at once — measured, eleven files took 715s serially against a 188s generation window to hide it in.	<code>qa_agent/session.py</code>
The author runs what it writes	After a batch is written, <code>`_verify`</code> runs one batched vitest invocation over just those files, then fixes them ONE FILE AT A TIME in a fresh conversation each, up to three rounds, four files in parallel. Batching the run is for vitest's ~2s startup; not batching the fixes is because a round handed nine failures at once left seven invented test ids in one file untouched while it worked on the others.	<code>qa_agent/author.py</code>
Best-version retention and signature stalling	Each fix round records <code>`(failure_count, body)`</code> ; a round that makes things worse is reverted to the better version, and two rounds producing the identical sorted failure signature stop the loop instead of repeating it.	<code>qa_agent/author.py</code>
The Python-written harness	<code>`vitest.config.mjs`</code> (jsdom, globals, <code>`@/`</code> alias repeated because Vitest does not read jsconfig.json, postcss neutered, esbuild <code>`jsx: 'automatic'`</code> so no <code>@vitejs/plugin-react</code> , 10s test and hook timeouts, <code>`tests/quarantine/**`</code> excluded), <code>`tests/setup.js`</code> (jest-dom, cleanup, and a replaceable <code>`window.location`</code> because jsdom's is non-configurable), and five helper modules.	<code>qa_agent/harness.py</code>
An in-memory Mongo that throws rather than lies	<code>`mongoMock.js`</code> implements <code>`\$eq/\$ne/\$in/\$nin/\$gt/\$gte/\$lt/\$lte/\$exists/\$regex/\$not/\$all/\$size/\$elemMatch, \$or/\$and/\$nor`</code> , and the update operators <code>`\$set/\$unset/\$inc/\$mul/\$min/\$max/\$push/\$addToSet/\$pull/\$pop/\$rename/\$currentDate`</code> , plus a cursor that actually sorts, <code>`updateMany`</code> , <code>`bulkWrite`</code> , <code>`findOneAndUpdate`</code> with the driver's before/after semantics, and <code>`distinct`</code> . Anything unimplemented throws a named error saying the mock is the limitation, not the route.	<code>qa_agent/harness.py</code>
Mechanical repairs on the way to disk	<code>`write_test_file`</code> runs three Python passes before writing: <code>`drop_redundant_mocks`</code> deletes a <code>`vi.mock`</code> for <code>`next/link`</code> or <code>`lucide-react`</code> (which would override the config's alias and lose the default export), <code>`ensure_mocks`</code> adds the <code>`vi.mock`</code> lines the file under test cannot run without, and <code>`add_helper_imports`</code> adds the import for any helper the test calls — a missing <code>`oid`</code> import was 8 of 15 failures in one measured run.	<code>qa_agent/session.py</code>
The manifest	<code>`.agentforge/qa/manifest.json`</code> maps each test path to <code>`{target, phase, tier, stale}`</code> and is the only thing that lets a failure be attributed to an application file — without it the fixer would be handed the whole project. Loaded on <code>`bind()`</code> , and <code>`adopt_orphans`</code> registers tests already on disk by inverting <code>`test_path_for`</code> .	<code>qa_agent/session.py</code>

Feature	What it does	Where
Staleness marking	After the analyzer's repair pass, <code>mark_stale</code> flags every test whose target's bytes changed, so the bug fixer sees "out of date" as a prior rather than as evidence about the code.	<code>qa_agent/session.py</code> , <code>server.py</code>
Failure classification, diagnosis and DOM extraction	<code>classify</code> assigns IMPORT/SYNTAX/TIMEOUT/ASSERTION/RUNTIME; <code>diagnose</code> names thirteen measured patterns ("route handler crashed", "seed-data ambiguity", "invented testid", "vi.mock hoisting", ...); <code>split_dom</code> pulls Testing Library's printed markup out of the blob, collapses SVGs and long class attributes and caps it at 1600 chars so it reaches the model instead of being truncated away.	<code>qa_agent/runner.py</code>
Written hints for the two mute failure classes	A <code>STACK_TRACE_ERROR</code> timeout and an <code>expected ... to be { Object (buffer) }</code> comparison both fail with messages that name nothing. <code>timeout_hint</code> and <code>objectid_hint</code> supply the three known causes in order; they travel in <code>TestFailure.hint</code> , not appended to the stack, because the stack is cut at 2000 chars and prompt builders keep only lines starting <code>at`</code> .	<code>qa_agent/runner.py</code> , <code>qa_agent/tests/test_timeout_hint.py</code>
The repair rounds	Up to <code>MAX_QA_FIX = 4</code> rounds inside a <code>QA_DEADLINE</code> of 1800s. Each round groups failures by test file, snapshots the test file and its target, dispatches four <code>BugFixerAgent.fix</code> calls in parallel, and re-runs only the touched files. A round that does not strictly shrink the failure set is reverted from the snapshot; two such stalls escalate one repair tier (0 usual, 1 refusal reason plus more context, 2 permission to fix the component, 3 the harness itself as a suspect).	<code>server.py:run_qa_unit_stage</code>
Code fixes must survive a rebuild	When a round wrote application code, <code>run_build_fix_loop</code> reruns; if the build is not green the snapshot restores every file the round touched and the code-fix count is decremented.	<code>server.py:run_qa_unit_stage</code>
The pass-rate floor	<code>_record_round_one</code> reports round one as a rate rather than a count — a count means different things in a 20-case and an 80-case suite — names the dominant failure classes, warns below <code>ROUND_ONE_FLOOR = 90</code> , and appends one JSON line to <code>.agentforge/qa/history.jsonl</code> with cases, passed, failed, rate, floor and the top five classes. It is a reported floor, not a gate: nothing is blocked by it.	<code>server.py:_record_round_one</code>
Quarantine	<code>VitestRunner.quarantine</code> moves a test AgentForge could not make trustworthy to <code>tests/quarantine/<name></code> with a <code>// QUARANTINED by AgentForge: <why></code> header, drops it from the manifest and records it on the report. Kept rather than deleted so the mistake can be read; the config's <code>exclude</code> already covers the directory. In the unit stage it is reached only when the bug fixer's verdict asks for it.	<code>qa_agent/runner.py</code> , <code>server.py</code>
Anti-cheat accounting	<code>runner.skipped()</code> lists cases marked <code>it.skip</code> that never ran and <code>runner.vanished(baseline)</code> lists cases that existed at the start of the stage and are gone — both reported as not passing, and <code>out["clean"]</code> requires neither. <code>_leave_unresolved</code> records every still-red case on the report under its own name with its diagnosis, and explicitly refuses to silence anything.	<code>qa_agent/runner.py</code> , <code>server.py:_leave_unresolved</code>

Feature	What it does	Where
Housekeeping around the suite	<code>`drop_tests_for_missing_targets`</code> deletes tests whose subject was removed or reduced to comments before the suite runs; <code>`_backfill_tests`</code> authors tests for every testworthy file the per-phase cap never reached; <code>`_remeasure_unit`</code> re-runs the whole suite at the very end on the bytes that are actually shipping and, if the later stages broke cases, gives two <code>`_repair_after_stages`</code> rounds with every failure pre-marked stale.	<code>server.py</code>
The scenario grammar	Nine verbs (GOTO, FILL, CLICK, WAIT_FOR, EXPECT_TEXT, EXPECT_URL, EXPECT_NO_ERROR, plus FLOW/AS headers), five selector kinds (role+name, label, placeholder, text, testid), 24-step cap. An unrecognised verb is dropped rather than executed, so model-written code is never run in AgentForge's process. Parse-time refusals include a GOTO leaving the app, a dynamic-route path with <code>`[`</code> in it, a regex using Python-only syntax, and re-querying a control that was already clicked.	<code>qa_agent/flows.py</code>
One scenario, two runners that cannot diverge	<code>`to_playwright_js`</code> emits <code>`tests/e2e/<slug>.spec.js`</code> plus a <code>`playwright.config.js`</code> from the parsed <code>`Scenario`</code> , not from the model's text, and <code>`E2EAgent._locator`</code> mirrors <code>`Selector.js_locator`</code> step for step. AgentForge runs the Python Playwright it already has (chromium 1223 already cached) — no npm install, no browser download; the spec is the deliverable.	<code>qa_agent/flows.py</code> , <code>qa_agent/e2e.py</code>
The signed-in sweep and the role-separation check	<code>`sweep`</code> opens every non-dynamic page (cap 25) on the context the flow just authenticated, catching faults the signed-out route probe never sees. <code>`role_separation`</code> signs in as each demo role through Better Auth's own <code>`/api/auth/sign-in/email`</code> , records which pages each role reached, and reports a page that opens for a role the rest of its section refuses — then re-checks over plain HTTP with redirects off before making the claim, and drops it if the page answers 3xx.	<code>qa_agent/e2e.py</code>
Results on disk, nine tabs, and a PDF	<code>`write_qa_report`</code> writes <code>`.agentforge/qa/report.json`</code> once at the end; <code>`read_qa_results`</code> gathers that plus <code>vitest.json</code> , <code>manifest.json</code> , <code>history.jsonl</code> , <code>lighthouse.json</code> and every file under <code>tests/unit</code> , <code>tests/e2e</code> and <code>tests/quarantine</code> , reporting a missing file as missing rather than as zero. The studio renders nine tabs from it and <code>`GET /api/qa-pdf/<project>`</code> renders the whole thing to A4 on request.	<code>server.py</code> , <code>studio/components/testing/</code> , <code>qa_agent/report_pdf.py</code>

3. Technology and tools

Technology	Version	What it does here	Why
Vitest	3 (`vitest@3`)	The unit runner. Invoked as `npx vitest run [paths] --reporter=json --outputFile=<rel>` with a 180s ceiling; config is `vitest.config.mjs`.	`.mjs` rather than `.js` because these generated projects have no `"type": "module"`, which would make a `.js` config containing `import` ambiguous.
jsdom	25 (`jsdom@25`)	The DOM environment for every unit test.	Chosen with eyes open about its limits — it does not navigate and does not run default form submission, and both facts are encoded in the author's prompt and in dedicated static checks.
@testing-library/react	16	Rendering and querying components under test.	The prompt pushes role/text queries and forbids class and style assertions: on a real build every remaining component-test failure was an exact heading, a Tailwind class, or a decorative string, and not one was a real defect.
@testing-library/jest-dom	6	The `toBeInTheDocument`/`toBeVisible`/`toBeDisabled` matchers, imported once in `tests/setup.js` as `@testing-library/jest-dom/vitest`.	Installed as a dev dep so the matchers exist without each generated test importing them.
@testing-library/user-event	14	Installed with the rest of the harness.	Present but actively discouraged for submits — measured, `userEvent.click` on a submit button timed out having called nothing, exactly like `fireEvent.click`; only `fireEvent.submit` reached fetch, in 16ms.
esbuild (via Vite)	—	JSX transform, configured as `esbuild: { jsx: 'automatic', jsxImportSource: 'react' }`.	Explicitly instead of `@vitejs/plugin-react` — one fewer dependency and one fewer Babel toolchain in a project that already ships SWC.
mongodb (the real driver)	—	Only `ObjectId`, imported by `mongoMock.js` and `request.js` so `oid()` produces a genuine 24-char id.	`new ObjectId('123')` throws inside the driver three frames from anything the model wrote, so the helper exists to make a hardcoded short id impossible.
Playwright (Python)	playwright==1.60.0, chromium build 1223	Drives the end-to-end scenario, the signed-in sweep and the role-separation check, headless at 1280×800.	The chromium revision is already in `~/AppData/Local/ms-playwright` from the tester, so the run costs no npm install and no browser download; `npx playwright test` would cost ~50MB per project and might want an uncached revision.
@playwright/test	—	Not run by AgentForge — the target of the emitted `tests/e2e/<slug>.spec.js` and `playwright.config.js`, which the user can run with `npm i -D @playwright/test && npx playwright test`.	The spec is the deliverable and the Python path is the runner; both are generated from the same parsed `Scenario` so they cannot drift.
Lighthouse	12 (`lighthouse@12` via `npx --yes`)	One audit of `http://127.0.0.1:<DEV_PORT>` writing `.agentforge/lighthouse.json`, 300s ceiling, `--chrome-flags="--headless=new --no-sandbox"`.	Driven through the Chromium Playwright already downloaded, located by `_playwright_chromium()` and passed as `CHROME_PATH`, so there is no second browser to install.
requests	—	Warms each page the flow will visit before it runs, and performs the redirects-off HTTP re-check that can retract a role-separation finding.	The re-check must not share a code path with the measurement it is checking — "the two measurements share only the app".

Technology	Version	What it does here	Why
ReportLab	—	Renders the PDF test report: A4 `BaseDocTemplate`, cover + `TableOfContents` via `multiBuild`, eight numbered sections and an appendix of test sources.	Pure-Python, so it works on Windows with no native libraries; and it is deliberately the same palette, table and badge furniture as the SRS PDF because the two land in the same inbox.
Python threading / queue / ThreadPoolExecutor	—	Four author workers, four fix workers in the author's verify loop, four in the unit stage's repair round, and an `NPM_LOCK` RLock serialising every npm invocation and every vitest run.	Without the install lock the first four author workers would each see `_runner_ready` False and run a concurrent `npm install` into the same node_modules.
Better Auth	—	Its `/api/auth/sign-in/email` endpoint is called with `page.evaluate` to switch roles for the role-separation check.	Driving the login form four times turns a fast check into a slow one and couples it to whatever the login page looks like this build.
Next.js App Router	16	The subject under test. Everything the harness compensates for is a property of it: async Server Components, `params` as a promise since Next 15, `redirect()`/`notFound()` throwing, `next/link` needing a client router.	Next's own bundled documentation at `node_modules/next/dist/docs/01-app/02-guides/testing/vitest.md` is what rules out unit-testing pages and sends them to E2E.
React 19 + Next 16 studio UI	—	The nine-tab Testing view, with Zustand for `qaReport`, Tailwind for styling and lucide-react icons; results are fetched from `/api/qa/<project>` and re-fetched whenever the live run stops.	`emit()` is fire-and-forget with no history and no replay, so a page opened after a build must read the files rather than the socket.

4. How it works

1. 1. Bind, without touching the build

``server.py`` constructs ``QASession(proj_dir, callbacks=_qa_callbacks(), model=qa_model, enabled=is_cloud_model(qa_model))``, wraps ``cb["on_phase"]`` and ``cb["on_file_written"]`` with the session's versions, then constructs the ``ArchitectAgent`` with that callbacks dict and calls ``qa.bind(arch)``. Bind loads the manifest, adopts orphan tests already on disk, sets ``concurrent`` from ``is_cloud_model``, constructs the ``UnitTestAuthor``, and starts a daemon thread that installs the test runner ahead of the first job. QA is cloud-only: a local model turns off authoring, repair and the browser pass, and the log says so.

2. 2. Files land; batches are queued

Every generated ``*.js``/``*.jsx`` write pushes a path into a buffer; at four the buffer flushes into the queue tagged with the plan phase it belongs to. ``_ensure_workers`` spins up to ``QA_AUTHOR_WORKERS = 4`` threads. Each job calls ``select_targets(files, self.read_source, ...)`` — reading from disk, never from ``arch.files`` — and ``ensure_runner()``, which under ``_runner_lock`` materialises the harness and runs ``npm install -D vitest@3 jsdom@25 @testing-library/...``.

3. 3. One model call per batch writes the tests

``UnitTestAuthor.write_for`` builds a two-message conversation: the ~200-line SYSTEM prompt, then the app idea (copied from ``arch.convo[1]``, never written back), the phase goal, and for each target its full source, its assigned test path, and a ``_preamble_note`` naming the exact ``vi.mock`` lines that file needs. The reply is parsed by ``FileStreamParser``; ``_accept`` rejects any path not in the assigned dict. Up to four ``<read_file .../>`` follow-ups are served through the analyzer, and a missed target gets one explicit re-ask.

4. 4. Three Python passes, then disk

``write_test_file`` checks the path against ``TEST_PATH_RE`` (``tests/unit/**/*.test.js[x]`` only), then runs ``drop_redundantmocks`` → ``ensuremocks`` → ``add_helper_imports``, writes the file, records ``{target, phase, tier, stale: False}`` in the manifest, saves ``manifest.json``, and fires ``on_file_written`` so the studio shows it. ``// SUSPECT:`` comments are then scraped out into ``report.suspects`` as findings for a human, never as failures.

5. 5. The author verifies its own batch

One ``vitest run`` over just the written paths (its report goes to ``.agentforge/qa/vitest-<sha1>.json``, keyed by the path set, so it never clobbers the full-suite ``vitest.json``). Unresolvable packages found in the failures are installed by ``install_missing``, which then deletes ``node_modules/.vite`` because Vite's pre-bundle would otherwise describe a tree that no longer exists. If most failures look like an unsettled module graph the run is retried after 3s, 6s and 10s. What is still failing is fixed one file at a time, in parallel, for up to three rounds.

6. 6. Drain, backfill, staleness

After generation, ``qa.drain(timeout=180)`` waits out the queue (the method's own default is 900s), ``_backfill_tests`` authors tests for every testworthy file the per-phase cap of four never reached, a ``FileSnapshot`` captures every manifest target, the analyzer's Seam A repair pass runs, and ``qa.mark_stale(qa_targets.changed())`` flags the tests whose subject moved underneath them.

7. 7. The unit stage

``run_qa_unit_stage`` runs between ``run_build_fix_loop`` and ``start_next`` — the only window where nothing holds ``next/``. It refuses to run at all unless the build is green. It materialises and installs the harness, drops tests whose target no longer exists, and then loops: run → record round one and the history line on round 1 → emit per-case results → if failures, group by file, snapshot test and target, dispatch ``BugFixerAgent.fix`` four-wide, re-run only the touched files.

8. 8. Arbitration and the write allowlist

``BugFixerAgent`` must emit ``VERDICT :: test|code :: <evidence>`` before any write block; the verdict is read from what has streamed so far, so a model that writes first and explains afterwards gets the safe default of ``test``, which may only write the test file. ``code`` may write only the manifest target. Weakening — ``it.skip``, deleted assertions, ``expect(true).toBe(true)`` — is checked in Python after the write and rejected. ``lib/mongodb.js`` is never editable.

9. 9. Reverting, stalling, escalating

A round that does not strictly shrink the failure set restores the snapshot and increments ``stalls``; at ``MAX_QA_STALLS = 2`` the tier goes up (0→3) and stalls reset; at tier 3 with no progress the loop calls ``_leave_unresolved`` and stops. If a round wrote application code, the build is re-checked and a red build restores exactly those files. The whole stage is bounded by ``QA_DEADLINE = 1800`s``.

10. 10. The final accounting

After the loop, if only targeted runs were done, one full-suite run gets the true numbers. Then ``skipped()`` and ``vanished(baseline)`` are reported as not-passing, ``_leave_unresolved`` records every remaining failure with its diagnosis on ``report.unresolved``, and ``out["clean"]`` is set only when all three are empty. Later, ``_remeasure_unit`` runs the suite once more on the shipping bytes and, if the E2E/security/runtime stages broke cases, gives two ``_repair_after_stages`` rounds with every failure pre-marked stale.

11. 11. The browser pass

``run_qa_e2e_stage`` runs after the tester loop has settled — that loop restarts the dev server between attempts, so a flow started inside it would be driving a server about to be killed. ``E2EAgent.author`` shows the model the route table, the demo-account roles and up to six pages of real markup (entry page, landing page, then shallowest routes first, 30k budget, each page carrying the components it imports). The scenario is parsed, emitted as a ``spec.js``, and run. All-selector failures buy one re-author; crashes go to ``_repair_runtime`` for up to 15 rounds, each followed by a rebuild, a snapshot revert on a broken build, and a dev-server restart.

12. 12. Sweep, security, performance, report

If the scenario completes, the same page object sweeps every non-dynamic route signed in (cap 25, deduped by first line of fault) and runs the role-separation check. Then ``analyzer.run_runtime`` (Seam C), ``run_security_stage`` (six deterministic code checks plus ``npm audit --json``), and ``run_perf_stage`` (Lighthouse). Finally ``write_qa_report`` puts unit, suite, security, e2e, performance and runtime into ``agentforge/qa/report.json``, which is the only place most of it exists — every stage's return value was previously discarded and ``emit()`` has no replay.

5. The code, file by file

Path	What lives there
qa_agent/___init___py	The public surface and the statement of the design: the build pipeline is untouched, `QASession` wraps one callback, and the repair half lives in `agents/bugfixer.py` because it is a bug-fixing agent like the others.
qa_agent/spec.py	What is worth testing, decided without a model. The four tier constants, `GENERATED`/`UNTESTABLE_LIB` exclusion sets, `MAX_PER_PHASE = 4`, `MAX_PER_PROJECT = 30`, `test_path_for`, `select_targets`, and the `TestTarget`/`TestFailure`/`QAReport` dataclasses.
qa_agent/session.py	Timing, queue, manifest. Callback wrappers, four author workers, `ensure_runner` under its own lock, `write_test_file` with the three mechanical passes, manifest save/load/adopt, `mark_stale`, `drain`. Also `requiredmocks`, `mock_line`, `ensuremocks`, `drop_redundantmocks`, `add_helper_imports` and the `HELPER_HOMES`/`HELPER_SPIES` tables.
qa_agent/author.py	1,368 lines, most of it the SYSTEM prompt — the mock contract, the nine measured mistakes, the selector rule, the `// SUSPECT:` escape hatch. Then `write_for`, `_verify`, `_fix_one`, `_failure_digest`, `_fix_prompt`, `_accept`, and a family of static checks of which only `_unbalanced` is currently called (from `server.py`).
qa_agent/harness.py	Writes `vitest.config.mjs`, `tests/setup.js`, `nextLink.jsx`, `mongoMock.js`, `authMock.js`, `navMock.js` and `request.js` as Python string literals; `deps_present`/`install`/`install_missing`/`_drop_vite_cache`; the `NPM_LOCK` that serialises every npm and vitest invocation.
qa_agent/runner.py	`VitestRunner.run` (report to file, never stdout), `_parse`, `_rel` (resolve both sides before `relative_to`, or the manifest lookup silently misses), `classify`, `diagnose`, `diagnose_all`, `split_dom`/`_compact_dom`, `timeout_hint`, `objectid_hint`, `case_names`/`skipped`/`vanished`, and `quarantine`.
qa_agent/snapshot.py	116 lines that bound the cost of a wrong automated edit. Bytes not text (newline translation would break byte-identical restore on Windows), first-capture-wins across rounds, and `None` recorded for a path that did not exist so an undo can be a delete.
qa_agent/flows.py	The scenario grammar — `VERBS`, `SELECTOR_KINDS`, `parse_selector`, `parse_scenario`, `Scenario.is_runnable`, `to_playwright_js`, `_js_step`, `PLAYWRIGHT_CONFIG` and the `GRAMMAR` block that is spliced into the E2E system prompt.
qa_agent/e2e.py	`E2EAgent`: `accounts` from `.agentforge/plan.json`, `markup_for`/`_journey_pages`, `author`, `write_spec`, `run` with per-step dispatch, `_wait_hydrated`, `warm`, `sweep`, `role_separation`, `_still_open`, `_reach_table`, `_landed_on`, `_locator` (which must mirror `Selector.js_locator`).
qa_agent/report_pdf.py	`build_qa_pdf(qa, out_path, project=...)` — cover with verdict badge, TOC, and eight sections: Summary, Unit tests (per-file table, every case up to 400, failures in full), Test timeline, Integration, API contracts (from the manifest), Bug reports, Security, Performance, plus Appendix A with the test sources cut at 9000 chars with the cut marked. `_missing()` prints "this stage did not run" rather than zeros.
qa_agent/tests/test_split_dom.py	Regression test for the bug that shipped a broken navbar: the rendered DOM sat at the front of a 6230-char blob, `stack` was `blob[:2000]`, and prompt builders keep only `at` lines, so the model saw one sentence.
qa_agent/tests/test_timeout_hint.py	Proves the frozen-clock advice survives the trip to the model — it was previously appended to a 2365-char blob and cut off entirely, and would have been filtered as prose even intact.
qa_agent/tests/test_stream_deadlines.py	Two deadlines of different kinds: `stall` as the longest tolerated silence (a read timeout) and `timeout` as the total budget; the author previously passed neither and inherited a 1800-second default.

Path	What lives there
qa_agent/tests/test_heartbeat.py	The progress line has to distinguish a live stream from a dead one — it printed "0 reasoning token(s)" for four minutes against a model with no thinking mode.
server.py:read_qa_results	Gathers vitest.json, manifest.json, report.json, history.jsonl, lighthouse.json (scores rounded to percentages, five named metrics, fetchTime, runtimeError) and every file under tests/unit, tests/e2e and tests/quarantine. Exists because `/api/files/` cannot reach them: `.agentforge` is in `SKIP_DIRS` and `SRC_EXT` is only `{.js,.jsx,.css}`.
server.py:write_qa_report	Serialises unit, suite (passed/failures/unresolved/suspects/quarantined/written/skipped_reason), security (ran/findings/audit), e2e, performance and runtime into `.agentforge/qa/report.json`. Never fatal.
server.py:run_qa_unit_stage	The whole repair loop: four rounds, snapshot revert, two-stall tier escalation, the code-fix rebuild gate, the skipped/vanished accounting and the final `clean` flag.
server.py:_leave_unresolved	Replaced the behaviour that rewrote red cases as `it.skip` and moved files to quarantine — across the projects on disk that was 30 skipped cases and 32 set-aside files counted as passing.
server.py:_record_round_one	The pass rate, the dominant failure classes, the `ROUND_ONE_FLOOR = 90` warning and the append to `history.jsonl`.
server.py:run_perf_stage	Lighthouse 12 through `npx --yes` with `CHROME_PATH` set to Playwright's cached Chromium; stamps `measured_on: "next dev — not a production build"` onto the scores.
studio/components/testing/TestingResult.jsx	The nine-tab shell: tab list with count badges, retry-four-times load, reload whenever the live run stops, and the PDF button that fetches the blob (rather than linking) so a failure arrives as a log sentence instead of a page of JSON.
studio/components/testing/Overview.jsx	Six cards — Round one (rate against floor, top classes), The suite now (with explicit callouts for deleted and skipped cases), Left unresolved, Security, Performance, Runtime.
studio/components/testing/UnitTests.jsx	Collapsible per-file case list, failing files sorted first and auto-expanded, a `never ran` stat for skipped cases and the first six lines of each failure message.
studio/components/testing/Timeline.jsx	A bar per build's round-one rate with a dashed line at the floor, red below it, plus the same data as a table newest-first.
studio/components/testing/Security.jsx	The six check codes listed by name with what each looks for and clean/hit status, then each finding with severity, path, message and fix — and the `npm audit` summary line.
studio/components/testing/Coder.jsx	A file list and a source pane over every test on disk, with quarantined files tagged `set aside`.

6. Why it is built this way

Design decisions with their reasons. Where the source records the failure that forced a choice, that failure is given rather than a general principle.

Decision	Why
The model writes test bodies; Python writes the config and the mocks.	Every configuration hazard produces a failure that reads exactly like an application bug. Vitest does not read `jsconfig.json`, so without an explicit `resolve.alias` every test dies on `Failed to resolve import "@lib/mongodb" — which looks like a missing module in the app, and the bug fixer then goes after correct code.
`mongoMock.js` throws a named error on any operator, update verb or bulk method it does not implement.	It used to understand four operators and fall through to a string comparison. `{ price: { \$gt: 10 } }` stringified to `[object Object]`, matched nothing, the route returned an empty list, and the repair loop rewrote a correct test until it agreed with a broken mock. The same shape for absent methods: `updateMany` missing meant a `TypeError` inside the route's own try/catch, a 500 that reads as a database fault, and three repair rounds spent on correct code. Silence points at the app; a named error points at the mock.
`clone` keeps Date and ObjectId as objects; `serialize` is a separate function that stringifies.	Sharing one `JSON.parse(JSON.stringify(...))` for both is what hid the bug: a route doing `job.createdAt.toISOString()` died with "toISOString is not a function", its own try/catch turned that into a 500, and the test reported `expected 500 to be 200` — measured as every such failure in two separate builds.
`authMock.js` aliases `getCurrentUser`, `getUser`, `currentUser` and `requireUser` to the same reader.	A generated app often wraps `getSessionUser` behind its own name; without the aliases the import resolves to `undefined`, calling it throws, the route's catch returns 500, and every case fails as "expected 500 to be 401". Measured on one build: 32 of 46 failures, all of them this, across twelve route files.
`navMock.js`'s `redirect` and `notFound` are spies AND they throw.	Both halves are needed. A plain function cannot be asserted on (`[Function redirect] is not a spy`), and Next's real `redirect()` throws to abort the render — guard code assumes it never returns, so a mock that returns lets `if (!user) redirect('/login')` fall through to `user.role` and the test reports "Cannot read properties of null", which reads as a missing null check in correct code.
`postForm` builds a `URLSearchParams` body, not a `FormData`, and `postJson` forwards a `FormData` to it.	Building a Request with a FormData body sets no Content-Type under jsdom, so `request.formData()` throws "Content-Type was not one of multipart/form-data or application/x-www-form-urlencoded", the handler's catch returns 500, and the failure reads as the route crashing. And a FormData stringified by the JSON helper yields `{}` — measured, two cases in one file survived nine repair rounds and both escalation tiers against a route that was never wrong.
Tests live under `tests/` and are written by `QASession.write_test_file`, never `arch.write_file`.	Three measured reasons. `unresolved_packages()` scans every `./` in `arch.files` regardless of directory, so a test would make `vitest` and `jsdom` look like missing imports and burn an LLM turn installing them mid-build; `_verify_output()` counts files outside the scaffold and tests would mask an empty build; and `write_file` unlinks the twin stem, so writing `x.test.jsx` would delete an existing `x.test.js`.
The author runs the tests it just wrote instead of statically checking them.	Stated in `_verify`: every failure class chased with a new static check — an invented `data-testid`, a role the markup lacks, a hoisting error, a too-short `waitFor`, a seeded Date in the wrong timezone — says exactly what is wrong the first time the test is run, and reading it generalises to the classes nobody has hit yet. The static checks remain in the file but `_accept` sets `advice = ""` unconditionally, so the fix prompt's "## A static check says" slot is never populated; only `_unbalanced` is still used, and `server.py:_report_unparseable` points it at application code instead.
Fixes are done one file per conversation, but the run that finds them is batched.	vitest costs ~2s of startup whatever it executes, so batching the run is free. Batching the fixes is not: asking for nine corrected files in one turn produced nine mediocre ones — measured, a round that handed back every failure at once left seven invented `data-testid`s in one file untouched while it worked on the others.

Decision	Why
<code>`_failure_digest`</code> groups failures by identical message.	A file that forgot one <code>`vi.mock`</code> fails every case with the same text. Listing it five times spends the prompt on repetition and makes five separate-looking problems out of one — measured as the exact shape of 39% of all first-run failures. "5 cases, all of them this" is also the more useful sentence, because it says the fault is in the setup.
Nothing red is ever silenced — no <code>`it.skip`</code> rewriting, no automatic quarantine at the end of the loop.	Both used to happen, and every exit from the repair loop then ended in the words "the suite is green". Across the projects on disk that was 30 skipped cases and 32 set-aside files counted as passing. Deleting a case removes it from both sides of the ratio, so the percentage improves precisely because the evidence left — and <code>`agents/bugfixer.py`</code> already refuses a repair that introduces <code>`it.skip`</code> , so AgentForge has to obey the rule it enforces on the model.
The round-one pass rate is recorded as a rate with a 90% floor, and the floor blocks nothing.	A count does not travel — "5 failing" means different things in a 20-case and an 80-case suite, and says nothing about why. Every authoring fix this stage has had came from classifying failures by hand across every project on disk after the fact; the history line puts that answer on screen for every build so the next regression announces itself.
A role-separation finding is re-measured over plain HTTP with redirects off before it is reported.	Re-running the same navigation would re-run its bug with it. The first measurement is one reading of <code>`page.url`</code> after one navigation, and one flaky reading accuses a page that is correctly guarded — measured on a nursery build, <code>`/grower/add-plant`</code> was reported open to the owner and answered 307 to <code>`/`</code> every time over HTTP. Anything that stops the re-check keeps the finding, so it can only ever remove a claim it actively disproved.
The Lighthouse score is stamped <code>`measured_on: "next dev — not a production build"`</code> and blocks nothing.	A slow page is a judgement call, not a defect, and a build that refused to finish over a performance score would be refusing to finish over a hero image somebody wanted. The dev server compiles each route on first request and serves unminified JavaScript, so the number is a floor: the built app can only be faster. The PDF and the Performance tab both repeat the caveat next to the number.

7. Interfaces

Name	Shape
GET /api/qa/<project>	<code>`read_qa_results`</code> → <code>`{project, have:{vitest,manifest,report,history,performance,tests}, vitest, manifest, report, history[], performance:{scores,metrics,fetchTime,runTimeError}, tests:{path: source}}`</code> . 404-shaped <code>`{error}`</code> for an unknown project.
GET /api/qa-pdf/<project>	Renders <code>`build_qa_pdf`</code> on demand to <code>`agentforge/qa/Test_Report.pdf`</code> and returns <code>`application/pdf`</code> with <code>`Content-Disposition: attachment; filename="<proj>-test-report.pdf"`</code> ; <code>`{error}`</code> with 500 on failure. Rendered per request because the testing tab keeps changing as tests are repaired.
<code>.agentforge/qa/manifest.json</code>	<code>`{ "tests/unit/**/x.test.js": {target, phase, tier, stale, adopted?} }`</code> — the only mapping from a failing test back to the app file it is about.
<code>.agentforge/qa/vitest.json</code>	Vitest's own JSON reporter output for a full-suite run: <code>`numPassedTests`, `numFailedTests`, `numTotalTests`, `numTotalTestSuites`, `startTime`, `testResults[].assertionResults[][{fullName,title,ancestorTitles,status,duration,failureMessages}]`</code> .
<code>.agentforge/qa/vitest-<sha1>.json</code>	The same shape for a targeted run, keyed by a 10-char sha1 of the sorted path list, so the author's per-file runs never clobber the full-suite report.
<code>.agentforge/qa/report.json</code>	<code>`{at, unit:{ran,passed,failed,fixed,code_fixes,skipped?,deleted?,orphaned?,clean,final_check?}, suite:{passed,failures[],unresolved[],suspects[],quarantined[],written[],skipped_reason}, security:{ran,findings[],audit}, e2e:{ran,flow,passed,failed,fixed,failures[]}, performance:{}, runtime:[]}`</code> .
<code>.agentforge/qa/history.jsonl</code>	One JSON object appended per build: <code>`{at, cases, passed, failed, rate, floor, top:[{class,count}]}`</code> .
<code>.agentforge/lighthouse.json</code>	Raw Lighthouse 12 output; <code>`read_qa_results`</code> projects <code>`categories.*.score × 100`</code> and five audits (LCP, CLS, TBT, FCP, speed-index) into <code>`performance`</code> .
<code>vitest.config.mjs</code>	Written by the harness. <code>jsdom</code> , <code>globals</code> , <code>`setupFiles: ['./tests/setup.js']`</code> , <code>`include: ['tests/unit/**/*test.{js,jsx}']`</code> , <code>`exclude: ['**/node_modules/**','tests/quarantine/**']`</code> , <code>`testTimeout`/`hookTimeout` 10000</code> , <code>`restoreMocks`</code> , <code>`@`</code> and <code>`next/link`</code> aliases, <code>postcss</code> disabled, <code>esbuild</code> automatic JSX.
<code>tests/helpers/mongoMock.js</code>	Exports <code>`getCollection`</code> , <code>`getDb`</code> , <code>`serialize`</code> , <code>`ObjectId`</code> , a default resolved client, and the test surface <code>`__seed(name, rows)`</code> , <code>`__reset()`</code> , <code>`__all(name)`</code> , <code>`oid(hex?)`</code> .
<code>tests/helpers/authMock.js</code>	<code>`getSessionUser`</code> (a <code>`vi.fn`</code>), <code>`auth.api.getSession`</code> , the aliases <code>`getCurrentUser`/`getUser`/`currentUser`/`requireUser`</code> , a default export bundling them, and <code>`__setUser(user null)`</code> which normalises <code>`id`</code> to a string and deletes <code>`_id`</code> .
<code>tests/helpers/navMock.js</code>	<code>`useRouter`/`usePathname`/`useParams`/`useSearchParams`</code> , spies <code>`push`</code> , <code>`replace`</code> , <code>`back`</code> , <code>`forward`</code> , <code>`refresh`</code> , <code>`prefetch`</code> , throwing spies <code>`redirect`/`notFound`</code> , and <code>`__setPath(path, {params, query})`</code> / <code>`__resetNav()`</code> .
<code>tests/helpers/request.js</code>	<code>`oid(hex?)`</code> , <code>`postJson(handler, body, opts)`</code> , <code>`postForm(handler, body, opts)`</code> , <code>`getJson(handler, url, opts)`</code> , <code>`patchJson`/`putJson`/`deleteJson`</code> . All return <code>`{status, json, res}`</code> ; the handler is called as <code>`handler(request, {params: Promise.resolve(params)})`</code> .
<code>tests/unit/**</code> — the authored tests	Path derived by <code>`test_path_for`</code> : routes → <code>`tests/unit/api/<flattened path>.test.js`</code> , components → <code>`tests/unit/components/<Stem>.test.jsx`</code> , libs → <code>`tests/unit/lib/<stem>.test.js`</code> . The extension follows the target so the JSX transform is never in doubt.

Name	Shape
tests/quarantine/** – set aside, never run	The original file prefixed with `// QUARANTINED by AgentForge: <why>`. Excluded by the vitest config; still read back by `read_qa_results` and shown in the Coder tab tagged `set aside`.
tests/e2e/<slug>.spec.js + playwright.config.js	A real `@playwright/test` spec generated from the parsed `Scenario` (never from the model's text), collecting `pageerror` and console errors into `problems`, navigating with `waitUntil: 'load'` and asserting with 15s timeouts. `BASE_URL` overridable; config `testDir: './tests/e2e'`, 60s timeout, no retries, `trace: 'off'`.
The scenario grammar (model → AgentForge)	`FLOW ::`, `AS ::`, then `GOTO :: /path`, `FILL :: <selector> :: <value>`, `CLICK :: <selector>`, `WAIT_FOR :: <selector>`, `EXPECT_TEXT :: /re/i`, `EXPECT_URL :: /re/`, `EXPECT_NO_ERROR`, `DONE`. Selectors: `role=X name=/re/i`, `label=`, `placeholder=`, `text=`, `testid=`. Values may use `{{email}}`/`{{password}}`/`{{role}}`, substituted from `.agentforge/plan.json`'s `demo_accounts`.
The repair verdict protocol (model → bugfixer)	`VERDICT :: test code :: <one sentence naming the evidence>`, emitted before any `<write_file>` block. `test` may write only the test file, `code` only the manifest target; missing or unparseable defaults to `test`.
Websocket events this subsystem emits	`{type:"test_result", status: pass fail warn skip, msg, detail}` per case and per Lighthouse category; `{type:"phase", phase, title, status}` with the QA phase ids -14 authoring/backfill, -15 unit run, -16 repair round, -17 end-to-end, -18 app repair, -23 performance; plus `log`, `file` and `command` events routed through `_qa_callbacks()`.
Studio store contract	`api.qa(project)` fills `useStore.qaReport`; `api.qaPdfUrl(project)` is fetched as a blob and downloaded as `<project>-test-report.pdf`. The nine tabs are Overview, Unit Testing, Test Timeline, Integration, API Contracts, Bug Reports, Security, Performance, Coder — each reading a different slice of the same payload and rendering "this stage has no record" rather than a zero when its slice is absent.